

EU AI ACT PACK

AI Governance Architecture

Oversight & decision-enforcement design

Lexy AI Hiring Platform · Generated 2026-07-20

Factual description of the system as built. Not legal advice; assumptions flagged for counsel.

Owner: Engineering · Compliance **Status:** v1 (T010) — schema split, enforcement, append-only events, jurisdiction policies, stub appeal **Cutover model:** Soft (option B). Existing rows keep `applications.stage`; new governance columns are additive. **Related:** `AI_SYSTEM_CARD.md`, `SOC2_CONTROLS.md`, `RUNBOOK_APPEAL_HANDLING.md`

1. Why this layer exists

Every modern AI-employment regulation that touches Lexy — NYC LL144, Colorado SB24-205 (Colorado AI Act), Illinois HRA AI amendments + AIVI, the EU AI Act (Annex III high-risk employment) — converges on a single non-negotiable principle:

A consequential adverse employment decision MUST be made by a named natural person, not by an autonomous model.

The previous schema conflated *funnel position* (`applications.stage`) with *who terminated consideration*. That conflation is legally indefensible and would not survive an LL144 independent audit or a CO Attorney General inquiry. The Governance Layer makes that distinction structural rather than procedural.

2. Schema split — the foundation

Migration `0016_governance_decision_split.sql`.

Two new enums, deliberately disjoint:

ai_recommendation_enum	final_decision_enum
advance, reject, hold, lapsed, flag_fraud, no_recommendation	human_advance, human_reject, human_hold, human_lapsed, human_hired, human_offer, candidate_withdrawn, legacy_pre_gate

`final_decision_enum` has **no `ai_*` values by design**. A misbehaving service cannot store an autonomous AI rejection in `final_decision` because the type system rejects it.

`applications` adds:

<code>ai_recommendation</code>	<code>ai_recommendation_enum</code>
<code>ai_recommendation_at</code>	<code>timestamptz</code>
<code>ai_recommendation_model</code>	<code>text</code>
<code>ai_recommendation_score</code>	<code>real</code>
<code>final_decision</code>	<code>final_decision_enum</code>
<code>final_decision_by</code>	<code>text</code> -- <code>user_id</code> , NOT NULL when <code>final_decision</code> is set
<code>final_decision_at</code>	<code>timestamptz</code>
<code>final_decision_attestation</code>	<code>text</code> -- standardised attestation copy
<code>final_decision_reason</code>	<code>text</code> -- structured rationale (code + free text)
<code>gated_by_jurisdiction</code>	<code>text[]</code>
<code>policy_version_id</code>	<code>text</code> -- which policy was in effect

Two CHECK constraints make the invariant DB-enforced:

```

CHECK ( final_decision IS NULL
      OR final_decision_by IS NOT NULL
      OR final_decision = 'legacy_pre_gate' );

CHECK ( final_decision IS NULL
      OR final_decision_attestation IS NOT NULL
      OR final_decision = 'legacy_pre_gate' );

```

The `legacy_pre_gate` value is reserved for any future backfill of pre-cutover rows; it is **never** written by application code.

`applications.stage` is **unchanged**. It still drives every existing dashboard query. The Governance Layer is additive.

3. Versioned, append-only policy

`jurisdiction_ai_policy_rules` carries one row per `(jurisdiction_code, scope, tenant_id, version)`. The set of currently-active rules is the slice where `effective_from <= now() < effective_to`.

Column	Notes
<code>scope</code>	<code>platform_floor</code> (mandatory) or <code>tenant_extension</code> (additive)
<code>gate_rejects</code> / <code>gate_lapsed</code> / <code>gate_holds</code>	OR-merged across all matching rules
<code>require_disclosure</code> / <code>require_appeal</code> / <code>require_audit</code>	OR-merged
<code>basis</code>	Free-text citation: LL144, CO SB24-205, IL HRA AI amendments + AIVI, EU AI Act (high-risk employment)
<code>effective_from</code> / <code>effective_to</code>	Versioning. <code>effective_to = NULL</code> means currently active.

Mutation safety:

- A `BEFORE UPDATE` trigger raises if anything except `effective_to` changes — supersede policies by inserting a new row and closing the prior `effective_to`.
- A `BEFORE DELETE` trigger blocks deletes outright.

`jurisdiction_disclosure_templates` mirrors this shape for candidate-facing notice copy (NYC AEDT, CO pre-decision, IL AIVI, EU AI Act). Legal owns the content; engineering owns the table shape.

Seeded at migration time:

Jurisdiction	Rule	Basis
US-NY-NYC	<code>gate_rejects</code> , <code>gate_lapsed</code> , <code>require_disclosure</code> , <code>require_audit</code>	NYC LL144
US-CO	+ <code>require_appeal</code>	CO SB24-205
US-IL	<code>gate_rejects</code> , <code>gate_lapsed</code> , <code>require_disclosure</code> , <code>require_audit</code>	IL HRA + AIVI
EU	+ <code>require_appeal</code>	EU AI Act Annex III

4. Centralised enforcement service

Every materially-adverse automated outcome MUST funnel through `artifacts/api-server/src/lib/governance/decision-enforcement.ts`. There are two entry points:

4.1 `evaluateAndApplyAi(input)` — AI-initiated paths

Used by every classifier, scorer, scheduler, or agent that produces an adverse recommendation.

```
1. loadContext(applicationId) → application + candidate + job
2. classifyJurisdictions(candidate.location, job.location) → string[]
3. resolveActivePolicy(jurisdictions, tenantId) → ResolvedPolicy
4. Decide `gated` = isAdverse && policy.gate<Intent>
5. UPDATE applications SET
    ai_recommendation, ai_recommendation_at, ai_recommendation_model,
    ai_recommendation_score, gated_by_jurisdiction, policy_version_id
    (final_decision is NEVER touched here)
6. recordDecisionEvent('decision_created', actor_kind='ai', ...)
7. If gated → recordDecisionEvent('policy_applied', ...)
8. Return { gated, jurisdictions, policyVersionId, blockLegacyStageWrite: gated }
```

The caller MUST honour `blockLegacyStageWrite: true` by *not* subsequently writing `applications.stage = 'rejected'`. The architect review greps for direct adverse writes outside the enforcement service.

4.2 `applyHumanDecision(input)` — human-initiated paths

Used by manual reject UIs, the human-review queue, and any future operator console.

```
1. loadContext(applicationId)
2. Validate attestation length (DB CHECK enforces non-null too)
3. UPDATE applications SET final_decision, final_decision_by,
    final_decision_at, final_decision_attestation, final_decision_reason
4. recordDecisionEvent('decision_reviewed' or 'decision_overridden',
    actor_kind=role, attestation=text, ...)
```

`wasOverride` is true iff the AI's prior recommendation maps to a different `final_decision` than the human chose. Recorded for litigation posture.

4.3 Jurisdiction classifier — default-gated

`classifyJurisdictions(candidate, job)` is intentionally a small, dependency-free, safe-by-default heuristic:

- Empty / null location → returns **all** platform-floor jurisdictions (gate everything; force human review). False positives are cheap (a review); false negatives are LL144 violations.
- “New York” (state) → still flag `US-NY-NYC` because we cannot rule out the five boroughs.
- “Remote” → `US-NY-NYC` + `US-CO` + `US-IL` (and EU unless explicitly US-only).

If precision ever matters more than the safe default, wire in a real geocoder.

5. Immutable audit events

decision_events is the table an LL144 auditor reads, the CO AG would subpoena, and SOC2 evidence draws from. It is append-only at the DB level:

```
CREATE TRIGGER decision_events_block_update_trg
  BEFORE UPDATE ON decision_events
  FOR EACH ROW EXECUTE FUNCTION decision_events_block_mutation();

CREATE TRIGGER decision_events_block_delete_trg
  BEFORE DELETE ON decision_events
  FOR EACH ROW EXECUTE FUNCTION decision_events_block_mutation();
```

Event types:

event_type	Written by
decision_created	evaluateAndApplyAi — AI produced a recommendation
policy_applied	evaluateAndApplyAi — gate fired in a regulated jurisdiction
decision_reviewed	applyHumanDecision — human confirmed the AI recommendation
decision_overridden	applyHumanDecision — human chose differently
appeal_requested	POST /appeals/:applicationId
appeal_completed	(T+1 — full appeal workflow)
disclosure_shown	UI hooks when a candidate is served a jurisdiction disclosure

Every event captures: actor_user_id, actor_kind, model_id, model_version, prompt_version, scoring_version, orchestration_version, policy_version_id, jurisdictions[], disclosure_version_id, payload jsonb.

The version fields are non-optional in intent — every emitter should populate the ones it knows. Future audits become impossible if these are missing, per the design directive.

6. HTTP surface

All routes mounted from artifacts/api-server/src/routes/governance.ts :

Method	Path	Auth	Purpose
POST	/api/applications/:id/human-decision	Bearer (recruiter / hiring_manager / tenant_admin / platform_admin)	Record an attested final decision. Body: { finalDecision, attestation, reasonCode?, reasonNotes? }
GET	/api/applications/pending-human-review	Bearer (same roles)	Tenant-scoped queue of applications with AI rec awaiting human
GET	/api/governance/jurisdiction-policies/active	Public (read-only)	Active policy rules for UI badges and procurement reviews
POST	/api/appeals/:applicationId	Optional (candidate can file unauthenticated)	Stub — records appeal + event, returns 202

Method	Path	Auth	Purpose
GET	/api/appeals	Bearer (recruiter+)	Admin queue stub

API tokens are explicitly rejected from `POST /human-decision` — the law requires a named natural person.

7. Wire-up — every adverse-decision site

The architect review verifies the following:

Site	Pattern
<code>routes/outreach.ts</code> DNC branch (<code>sentiment='do_not_contact'</code>)	<code>evaluateAndApplyAi({ intendedAction: 'reject' })</code> then continue if <code>blockLegacyStageWrite</code>
<code>routes/outreach.ts</code> negative branch (<code>sentiment='negative'</code>)	Same pattern
<code>routes/applications.ts</code> PUT (manual recruiter reject)	After <code>recordRejection</code> , also <code>applyHumanDecision</code> with standardised attestation
<code>routes/pipeline.ts</code> kanban application reject	Same

Pure recommendation functions (`intelligence.decideNextAction`) are unchanged — they return a recommendation rather than writing to the DB. The architecture is correct already at the function level.

8. UI

`/human-review` (recruiter portal) lists every pending application, shows AI recommendation + score + model id + gated jurisdictions, and provides:

- Multi-select bulk action
- Structured rationale dropdown (`insufficient_experience`, `role_mismatch`, ...)
- Free-text notes
- **Required** attestation checkbox with the design-spec wording:

“I reviewed the AI recommendations and role-relevant candidate information before confirming this action.”

- Confirm / Override (advance, hold, reject) buttons

Pragmatic bulk: a single attestation per batch, logged per-row server-side, so audit fidelity is preserved.

9. Known gaps (T+1 work)

- Full operational appeal workflow (assignment, SLA tracking, candidate communications, outcome propagation). See `RUNBOOK_APEAL_HANDLING.md`.

- Disclosure UI hooks — templates exist; surfaces in the candidate experience are not yet wired.
- Tenant-extension policy authoring UI (today: insert via SQL).
- Geocoder-backed jurisdiction classification.
- Anti-ghost engine → `ai_recommendation = 'lapsed'` routing (currently the engine flags for recruiter review but does not yet write through enforcement).

10. Auditor checklist

A regulator inspecting the dev database should be able to verify all of the following with simple SQL:

```
-- 1. No AI-authored final decisions, ever.
SELECT COUNT(*) FROM applications
  WHERE final_decision IS NOT NULL
        AND final_decision_by IS NULL
        AND final_decision <> 'legacy_pre_gate'; -- expect 0

-- 2. Every final decision carries an attestation.
SELECT COUNT(*) FROM applications
  WHERE final_decision IS NOT NULL
        AND final_decision_attestation IS NULL
        AND final_decision <> 'legacy_pre_gate'; -- expect 0

-- 3. Audit events cannot be mutated (try and fail).
UPDATE decision_events SET rationale = 'tampered' WHERE id = (SELECT id FROM decision_events LIMIT 1);
-- expect: ERROR: decision_events is append-only

-- 4. Active policies for a jurisdiction.
SELECT jurisdiction_code, basis, gate_rejects, gate_lapsed, require_appeal
  FROM jurisdiction_ai_policy_rules
 WHERE effective_to IS NULL
 ORDER BY jurisdiction_code;
```